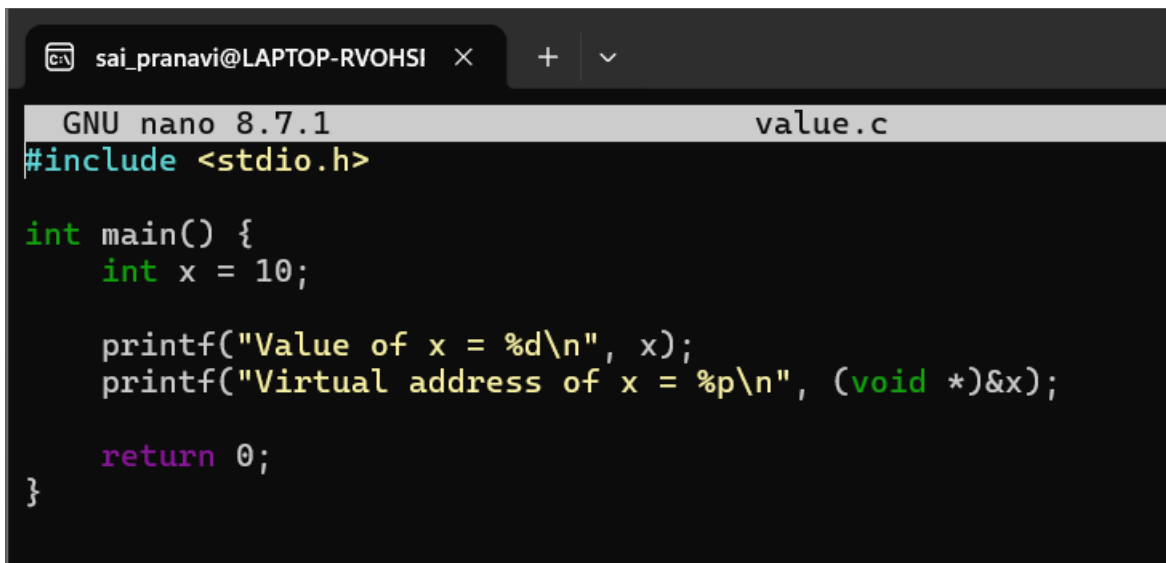


PRACTICAL CO4

1. Value and Virtual Address

```
sai_pranavi@LAPTOP-RVOHSPKQ:~$ nano value.c
sai_pranavi@LAPTOP-RVOHSPKQ:~$ gcc value.c -o value
sai_pranavi@LAPTOP-RVOHSPKQ:~$ ./value
Value of x = 10
Virtual address of x = 0x7ffe6923eaf4
```

Code:



```
GNU nano 8.7.1 value.c
#include <stdio.h>

int main() {
    int x = 10;

    printf("Value of x = %d\n", x);
    printf("Virtual address of x = %p\n", (void *)&x);

    return 0;
}
```

2. Code, Data, BSS, Heap, Stack Addresses

```
sai_pranavi@LAPTOP-RVOHSI  ×  +  v
GNU nano 8.7.1 stack.c
#include <stdio.h>
#include <stdlib.h>

int global = 10;
int bss_variable;

int main() {
    int local = 20;

    int *heap = malloc(sizeof(int));
    *heap = 30;

    printf("Code address    = %p\n", (void *)main);
    printf("Data address     = %p\n", (void *)&global);
    printf("BSS address        = %p\n", (void *)&bss_variable);
    printf("Heap address       = %p\n", (void *)heap);
    printf("Stack address      = %p\n", (void *)&local);

    free(heap);

    return 0;
}
```

Output

```
sai_pranavi@LAPTOP-RVOHSPKQ:~$ nano stack.c
sai_pranavi@LAPTOP-RVOHSPKQ:~$ nano stack.c
sai_pranavi@LAPTOP-RVOHSPKQ:~$ gcc stack.c -o stack
sai_pranavi@LAPTOP-RVOHSPKQ:~$ ./stack
Code address    = 0x571d7e1901a9
Data address     = 0x571d7e193010
BSS address      = 0x571d7e193018
Heap address     = 0x571daea17010
Stack address    = 0x7ffc7fb1da8c
```

3. Page Table

```
sai_pranavi@LAPTOP-RVOHSI  ×  +  ▾
GNU nano 8.7.1                                pagetable.c
#include <stdio.h>

int main() {
    int page_table[4] = {1, -1, 3, -1};

    int page;

    printf("Enter page number (0-3): ");
    scanf("%d", &page);

    if (page_table[page] == -1) {
        printf("Page Fault!\n");
        printf("Page must be loaded into memory.\n");
    } else {
        printf("Page is already in memory.\n");
        printf("Physical frame = %d\n", page_table[page]);
    }

    return 0;
}
```

Output:

```
sai_pranavi@LAPTOP-RVOHSPKQ:~$ nano pagetable.c
sai_pranavi@LAPTOP-RVOHSPKQ:~$ nano pagetable.c
sai_pranavi@LAPTOP-RVOHSPKQ:~$ gcc pagetable.c -o pagetable
sai_pranavi@LAPTOP-RVOHSPKQ:~$ ./pagetable
Enter page number (0-3): 1
Page Fault!
Page must be loaded into memory.
```

4. Virtual Address → Physical Address

```
sai_pranavi@LAPTOP-RVOHSI x + v
GNU nano 8.7.1 virtualadress.c
#include <stdio.h>

int main() {
    int page_table[] = {3, 5, 1, 7};
    int page_size = 100;

    int virtual_address = 250;

    int page = virtual_address / page_size;
    int offset = virtual_address % page_size;

    int frame = page_table[page];

    int physical_address = frame * page_size + offset;

    printf("Virtual Address = %d\n", virtual_address);
    printf("Page Number      = %d\n", page);
    printf("Offset           = %d\n", offset);
    printf("Frame Number       = %d\n", frame);
    printf("Physical Address = %d\n", physical_address);

    return 0;
}
```

Output:

```
sai_pranavi@LAPTOP-RVOHSPKQ:~$ nano virtualadress.c
sai_pranavi@LAPTOP-RVOHSPKQ:~$ gcc virtualadress.c -o virtualadress
sai_pranavi@LAPTOP-RVOHSPKQ:~$ ./virtualadress
Virtual Address = 250
Page Number      = 2
Offset           = 50
Frame Number     = 1
Physical Address = 150
```

5. Dynamic Memory Allocation

```
sai_pranavi@LAPTOP-RVOHSI x + v
GNU nano 8.7.1 memoryallocation.c
#include <stdio.h>
#include <stdlib.h>

int main() {
    int *p;

    p = malloc(5 * sizeof(int));

    if (p == NULL) {
        printf("Memory allocation failed\n");
        return 1;
    }

    for (int i = 0; i < 5; i++) {
        p[i] = (i + 1) * 10;
    }

    for (int i = 0; i < 5; i++) {
        printf("%d ", p[i]);
    }

    printf("\n");

    free(p);

    return 0;
}
```

Output:

```
sai_pranavi@LAPTOP-RVOHSPKQ:~$ nano memoryallocation.c
sai_pranavi@LAPTOP-RVOHSPKQ:~$ nano memoryallocation.c
sai_pranavi@LAPTOP-RVOHSPKQ:~$ gcc memoryallocation.c -o memoryallocation
sai_pranavi@LAPTOP-RVOHSPKQ:~$ ./memoryallocation
10 20 30 40 50
```

6. Null Pointer Dereferencing and Segmentation Fault

```
sai_pranavi@LAPTOP-RVOHSI  ×  +  ∨
GNU nano 8.7.1                Segmentation.c *
#include <stdio.h>

int main() {
    int *p = NULL;

    printf("Before accessing memory\n");

    *p = 100;

    printf("After accessing memory\n");

    return 0;
}
```

Output:

```
sai_pranavi@LAPTOP-RVOHSPKQ:~$ nano Segmentation.c
sai_pranavi@LAPTOP-RVOHSPKQ:~$ gcc Segmentation.c -o Segmentation
sai_pranavi@LAPTOP-RVOHSPKQ:~$ ./Segmentation
Before accessing memory
Segmentation fault ./Segmentation
```